

CURSO BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

IGOR DE SOUZA PINTO

RELATÓRIO DE ATIVIDADES

PINHAIS

2026

SUMÁRIO

1 CENÁRIO E REGRAS DE NEGÓCIO	3
2 FUNDAMENTAÇÃO E PLANEJAMENTO	4
2.1 Erro, defeito e falha	4
2.2 Verificação e validação	4
2.3 Plano de Teste	5
2.4 Níveis e tipos de teste	6
3 PROJETO DE CASOS DE TESTE	6
3.1 Particionamento de equivalência	7
3.2 Análise de valor limite	7
3.3 Tabela de decisão	8
3.4 Casos de teste formais	8
4 DESENVOLVIMENTO ORIENTADO A TESTES E AUTOMAÇÃO	10
4.1 Estratégia de desenvolvimento orientado a testes	10
4.2 Ciclos Red-Green-Refactor	11
4.3 Automação dos testes unitários	15
4.4 Cobertura de código	16
5 EXECUÇÃO, DEFEITOS E FERRAMENTAS	18
5.1 Execução dos testes	18
5.2 Relatórios de defeito	18
5.2.1 Defeito 1 - Acréscimo incorreto na entrega expressa	18
5.2.2 Defeito 2 - Limite máximo de peso incorreto	19
5.3 Comparação de ferramentas de teste	19
6 CONCLUSÃO	20

1 CENÁRIO E REGRAS DE NEGÓCIO

O módulo desenvolvido tem como objetivo calcular o valor de um frete a partir de três informações principais: peso da encomenda, distância do transporte e tipo de entrega. Para delimitar o comportamento esperado do sistema e permitir a aplicação das técnicas de teste propostas, foram definidas seis regras de negócio.

RN01 — Taxa base: O frete possui uma taxa base de R\$ 10,00.

RN02 — Limite de peso: O peso aceito deve estar entre 0,1 kg e 30 kg, inclusive.

RN03 — Limite de distância: A distância aceita deve estar entre 1 km e 1.000 km, inclusive.

RN04 — Fórmula do valor-base: O valor-base do frete é calculado por:

$\text{R\$ } 10,00 + (\text{peso} \times \text{R\$ } 2,00) + (\text{distância} \times \text{R\$ } 0,05)$

RN05 — Tipo de entrega: O tipo de entrega aplica um multiplicador ao valor calculado, sendo econômica = 1,00, padrão = 1,20 e expressa = 1,50.

RN06 — Validação de entradas: Peso, distância ou tipo de entrega inválidos devem ser rejeitados pelo sistema por meio de erro de validação.

A identificação das regras por meio dos códigos RN01 a RN06 facilita a rastreabilidade entre os requisitos definidos, os casos de teste elaborados e os testes automatizados implementados ao longo do trabalho.

2 FUNDAMENTAÇÃO E PLANEJAMENTO

2.1 Erro, defeito e falha

No contexto de testes de software, erro, defeito e falha representam conceitos relacionados, mas distintos. O erro corresponde a uma ação humana incorreta durante a especificação, implementação ou manutenção do software. Um exemplo aplicado ao módulo seria interpretar equivocadamente que a entrega expressa deve acrescentar 15% ao valor do frete, quando a regra definida estabelece acréscimo de 50%.

O defeito é a manifestação desse erro no artefato de software. No exemplo anterior, o defeito seria implementar no código um multiplicador de 1,15 para a entrega expressa em vez de

1,50. Já a falha ocorre durante a execução, quando o comportamento incorreto se torna observável. Assim, se um frete-base de R\$ 100,00 fosse calculado como R\$ 115,00 na modalidade expressa, em vez dos R\$ 150,00 esperados, estaria caracterizada uma falha.

2.2 Verificação e validação

A verificação busca confirmar se o software foi construído de acordo com as especificações definidas. No módulo de frete, isso envolve verificar se a fórmula, os limites de peso e distância e os multiplicadores de cada modalidade foram implementados conforme as regras de negócio documentadas.

A validação, por sua vez, procura confirmar se o comportamento do sistema atende ao uso esperado. Aplicada ao módulo, consiste em avaliar se, ao fornecer valores válidos de peso, distância e tipo de entrega, o sistema produz um valor de frete coerente, e se entradas inválidas são rejeitadas de forma adequada.

2.3 Plano de Teste

Item	Descrição
Escopo	Testar o módulo de cálculo de frete, abrangendo cálculo do valor-base, aplicação dos multiplicadores de entrega e validação de peso, distância e tipo de entrega.
Itens a testar	Função de cálculo de frete; limites de peso; limites de distância; modalidades econômica, padrão e expressa; tratamento de entradas inválidas.
Fora do escopo	Interface gráfica, integração com transportadoras reais, consulta por CEP, dimensões da embalagem, peso cúbico, pedágios, seguro, pagamento e rastreamento.
Tipos de teste	Testes funcionais, testes de regressão e testes de desempenho em um cenário hipotético de integração com um sistema completo.
Níveis de teste	Principalmente teste unitário no módulo atual; integração, sistema e aceitação são considerados para o cenário em que o módulo seja incorporado a uma aplicação completa.
CrITÉRIOS de entrada	Regras de negócio definidas, ambiente Python configurado, pytest e pytest-cov instalados e versão do código disponível para teste.

Critérios de saída	Todos os casos planejados executados; testes unitários aprovados; defeitos identificados registrados; cobertura de código medida e analisada.
Riscos	Implementação incorreta dos limites; multiplicadores incorretos; ausência de tratamento para entradas inválidas; interpretação equivocada dos resultados de cobertura.
Ambiente	Python 3.12, pytest, pytest-cov, execução local em Windows e versionamento com Git/GitHub.
Papéis	O aluno atua como desenvolvedor, projetista de testes e executor dos testes, por se tratar de atividade individual.

2.4 Níveis e tipos de teste

Teste unitário: É o nível principal utilizado neste trabalho, pois permite validar diretamente a função responsável pelo cálculo do frete e suas regras de validação de maneira isolada.

Teste de integração: Seria aplicado para verificar a comunicação do módulo com outras partes de uma aplicação, como uma API, um serviço de pedidos ou uma camada de persistência.

Teste de sistema: Seria utilizado para avaliar o fluxo completo de cálculo de frete dentro de uma aplicação já integrada, considerando a interação entre os componentes do sistema.

Teste de aceitação: Permitiria confirmar, sob a perspectiva do usuário ou responsável pelo negócio, se os valores calculados e as mensagens de erro atendem às regras esperadas.

Teste funcional: É aplicado para verificar as regras de negócio, incluindo cálculo por peso e distância, aplicação do tipo de entrega e rejeição de entradas inválidas.

Teste de regressão: É importante após alterações no código para confirmar que comportamentos anteriormente validados permanecem funcionando.

Teste de desempenho: Não é central na implementação acadêmica atual, mas seria relevante em um sistema real para verificar a capacidade do serviço de calcular grandes volumes de fretes em tempo adequado.

Teste de segurança: Possui importância secundária neste módulo isolado, mas em uma aplicação real ajudaria a garantir que entradas inesperadas ou malformadas não provoquem comportamentos indevidos.

3 PROJETO DE CASOS DE TESTE

O projeto dos casos de teste foi elaborado a partir das regras de negócio definidas para o módulo. Foram utilizadas as técnicas de particionamento de equivalência, análise de valor limite e tabela de decisão. Os casos formais foram organizados para cobrir cálculos válidos, fronteiras dos intervalos e entradas inválidas.

3.1 Particionamento de equivalência

O particionamento de equivalência divide o domínio de entrada em classes cujos valores devem produzir comportamentos equivalentes. Para este módulo, a técnica foi aplicada aos campos peso e distância.

Peso

Classe	Intervalo	Situação
P1	$\text{peso} < 0,1 \text{ kg}$	Inválido
P2	$0,1 \text{ kg} \leq \text{peso} \leq 30 \text{ kg}$	Válido
P3	$\text{peso} > 30 \text{ kg}$	Inválido

Assim, um valor representativo de cada classe pode ser utilizado para verificar o comportamento geral esperado daquela partição. Valores como 5 kg e 20 kg pertencem à mesma classe válida, enquanto valores abaixo de 0,1 kg ou acima de 30 kg pertencem a classes inválidas distintas.

Distância

Classe	Intervalo	Situação
D1	$\text{distância} < 1 \text{ km}$	Inválido
D2	$1 \text{ km} \leq \text{distância} \leq 1000 \text{ km}$	Válido
D3	$\text{distância} > 1000 \text{ km}$	Inválido

3.2 Análise de valor limite

A análise de valor limite foi aplicada aos mesmos campos porque defeitos são comuns nas fronteiras dos intervalos, especialmente em comparações que utilizam operadores como <, <=, > e >=. Foram selecionados valores imediatamente abaixo, no próprio limite e imediatamente acima das fronteiras definidas.

Valores-limite para peso

Valor	Resultado esperado
0,09 kg	Inválido
0,10 kg	Válido
0,11 kg	Válido
29,99 kg	Válido
30,00 kg	Válido
30,01 kg	Inválido

Valores-limite para distância

Valor	Resultado esperado
0 km	Inválido
1 km	Válido
2 km	Válido
999 km	Válido
1000 km	Válido
1001 km	Inválido

3.3 Tabela de decisão

A tabela de decisão foi construída com três condições principais: validade do peso, validade da distância e validade do tipo de entrega. As ações representam o cálculo normal ou a rejeição da entrada inválida. Nos casos em que mais de uma entrada é inválida, a implementação retorna o erro correspondente à primeira validação encontrada.

Condição / Regra	R1	R2	R3	R4	R5
Peso válido?	Sim	Não	Sim	Sim	Não
Distância válida?	Sim	Sim	Não	Sim	Não
Tipo válido?	Sim	Sim	Sim	Não	Não
Calcular frete	X				
Erro de peso		X			X

Erro de distância			X		
Erro de tipo				X	

3.4 Casos de teste formais

Foram definidos quatorze casos de teste formais. A quantidade supera o mínimo solicitado e mantém correspondência com os principais comportamentos cobertos pela suíte automatizada.

ID	Objetivo	Pré-condição	Passos	Dados de entrada	Resultado esperado	Prioridade
CT01	Calcular frete econômico	Sistema disponível e função de cálculo acessível	Informar os dados e executar o cálculo	peso=5; distância=100; tipo=econômica	R\$ 25,00	Alta
CT02	Calcular frete padrão	Sistema disponível e função de cálculo acessível	Informar os dados e executar o cálculo	peso=5; distância=100; tipo=padrão	R\$ 30,00	Alta
CT03	Calcular frete expresso	Sistema disponível e função de cálculo acessível	Informar os dados e executar o cálculo	peso=5; distância=100; tipo=expressa	R\$ 37,50	Alta
CT04	Validar peso mínimo aceito	Sistema disponível e função de cálculo acessível	Executar o cálculo no limite inferior válido	peso=0,1; distância=100; tipo=econômica	Cálculo realizado sem exceção	Alta
CT05	Validar peso máximo aceito	Sistema disponível e função de cálculo acessível	Executar o cálculo no limite superior válido	peso=30; distância=100; tipo=econômica	Cálculo realizado sem exceção	Alta
CT06	Rejeitar peso abaixo do mínimo	Sistema disponível e função de cálculo acessível	Executar o cálculo com peso abaixo do limite	peso=0,09; distância=100; tipo=econômica	ValueError	Alta
CT07	Rejeitar peso acima do máximo	Sistema disponível e função de	Executar o cálculo com peso	peso=30,01; distância=100;	ValueError	Alta

		cálculo acessível	acima do limite	tipo=econômica		
CT08	Validar distância mínima aceita	Sistema disponível e função de cálculo acessível	Executar o cálculo no limite inferior válido	peso=5; distância=1; tipo=econômica	Cálculo realizado sem exceção	Alta
CT09	Validar distância máxima aceita	Sistema disponível e função de cálculo acessível	Executar o cálculo no limite superior válido	peso=5; distância=1000; tipo=econômica	Cálculo realizado sem exceção	Alta
CT10	Rejeitar distância abaixo do mínimo	Sistema disponível e função de cálculo acessível	Executar o cálculo com distância abaixo do limite	peso=5; distância=0; tipo=econômica	ValueError	Alta
CT11	Rejeitar distância acima do máximo	Sistema disponível e função de cálculo acessível	Executar o cálculo com distância acima do limite	peso=5; distância=1001; tipo=econômica	ValueError	Alta
CT12	Rejeitar tipo de entrega inválido	Sistema disponível e função de cálculo acessível	Executar o cálculo com modalidade inexistente	peso=5; distância=100; tipo=urgente	ValueError	Alta
CT13	Calcular valor intermediário válido	Sistema disponível e função de cálculo acessível	Executar o cálculo com valores intermediários	peso=15; distância=500; tipo=econômica	R\$ 65,00	Média
CT14	Calcular combinação válida expressa	Sistema disponível e função de cálculo acessível	Executar o cálculo com combinação válida e modalidade expressa	peso=10; distância=200; tipo=expressa	R\$ 60,00	Média

Os casos CT04 a CT11 evidenciam as fronteiras válidas e inválidas dos campos peso e distância. Os casos CT01 a CT03 verificam os multiplicadores das modalidades de entrega,

enquanto CT12 verifica a rejeição de um tipo inexistente. Os casos CT13 e CT14 ampliam a cobertura com combinações válidas adicionais.

4 DESENVOLVIMENTO ORIENTADO A TESTES E AUTOMAÇÃO

4.1 Estratégia de desenvolvimento orientado a testes

O módulo foi desenvolvido seguindo a abordagem Test-Driven Development (TDD), na qual cada nova funcionalidade foi conduzida por ciclos Red-Green-Refactor. Na etapa Red, um teste foi escrito antes da funcionalidade correspondente e executado com resultado de falha. Na etapa Green, foi implementado somente o necessário para fazer os testes passarem. Por fim, na etapa Refactor, a estrutura do código foi melhorada sem alterar o comportamento já validado.

O histórico do repositório Git foi utilizado como evidência dos três ciclos realizados, permitindo rastrear separadamente os commits de teste, implementação e refatoração. Para complementar a rastreabilidade, os estados históricos dos commits foram reexecutados e registrados em capturas do terminal, apresentadas nas figuras a seguir.

```
PS C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calcula-frete-testes-software> git log --oneline --reverse
22b6292 Initial commit
4047913 Add initial project structure with .gitignore, README, requirements, and source/test files
6a0df5a test: adiciona teste inicial de frete economico
936eaa4 feat: implementa calculo basico de frete economico
c148674 refactor: extrai constantes do calculo de frete
fafd6d6 test: adiciona testes para entregas padrao e expressa
5f55e09 feat: adiciona tipos de entrega padrao e expressa
258c646 refactor: centraliza multiplicadores de entrega
bfb4e4c test: adiciona validacoes de peso distancia e tipo de entrega
9d6b921 feat: adiciona validacoes ao calculo de frete
5cce368 refactor: organiza validacoes e limites do frete
6ee0296 (HEAD -> main, origin/main, origin/HEAD) test: adiciona testes parametrizados de valores limite
PS C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calcula-frete-testes-software>
```

Figura 1 – Histórico de commits utilizado para rastrear os ciclos Red-Green-Refactor.

4.2 Ciclos Red-Green-Refactor

Ciclo	Etapa	Commit	Evidência / alteração
1	RED	6a0df5a	Teste inicial para frete econômico. A execução falhou porque calcular_frete ainda não existia.

1	GREEN	936eaa4	Implementação mínima do cálculo econômico. O teste passou.
1	REFACTOR	c148674	Extração das constantes TAXA_BASE, VALOR_POR_KG e VALOR_POR_KM sem alterar o resultado.
2	RED	fafd6d6	Inclusão dos testes para entregas padrão e expressa. Os dois novos testes falharam.
2	GREEN	5f55e09	Implementação dos multiplicadores de entrega padrão e expressa. Os três testes passaram.
2	REFACTOR	258c646	Centralização dos multiplicadores no dicionário MULTIPLICADORES ENTREGA.
3	RED	bfb4e4c	Inclusão de testes para peso, distância e tipo de entrega inválidos. Resultado: 3 testes aprovados e 5 falhas.
3	GREEN	9d6b921	Implementação das validações com ValueError. Resultado: 8 testes aprovados.
3	REFACTOR	5cce368	Extração dos limites para constantes e das validações para a função auxiliar validar entrada.

No primeiro ciclo foi implementado o cálculo básico da modalidade econômica. O segundo ciclo adicionou os acréscimos das modalidades padrão e expressa. O terceiro ciclo concentrou-se na validação dos limites de peso e distância e na rejeição de tipos de entrega não

reconhecidos. Dessa forma, os ciclos evoluíram gradualmente a regra de negócio sem antecipar funcionalidades. As capturas das Figuras 2 a 8 apresentam a reexecução dos estados Red e Green correspondentes aos commits históricos de cada ciclo.

```
PS C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software> python -m pytest
===== test session starts =====
platform win32 -- Python 3.12.0, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software
plugins: cov-7.1.0
collected 0 items / 1 error

===== ERRORS =====
ERROR collecting tests/test_frete.py
ImportError while importing test module 'C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software\tests\test_frete.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\..\AppData\Local\Programs\Python\Python312\Lib\importlib\__init__.py:90: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
tests\test_frete.py:1: in <module>
    from src.frete import calcular_frete
E ImportError: cannot import name 'calcular_frete' from 'src.frete' (C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software\src\frete.py)

===== short test summary info =====
ERROR tests/test_frete.py
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
===== 1 error in 0.17s =====
PS C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software>
```

Figura 2 – Ciclo 1, etapa Red: o teste inicial falha porque a função `calcular_frete` ainda não existe.

```
PS C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software> python -m pytest
===== test session starts =====
platform win32 -- Python 3.12.0, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software
plugins: cov-7.1.0
collected 1 item

tests\test_frete.py . [100%]

===== 1 passed in 0.03s =====
PS C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software>
```

Figura 3 – Ciclo 1, etapa Green: o cálculo econômico é implementado e o teste passa.

```
PS C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software> python -m pytest
===== test session starts =====
platform win32 -- Python 3.12.0, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software
plugins: cov-7.1.0
collected 3 items

tests\test_frete.py .FF [100%]

===== FAILURES =====
test_calcular_frete_padrao

def test_calcular_frete_padrao():
> assert calcular_frete(5, 100, "padrao") == 30.00
E AssertionError: assert None == 30.0
E + where None = calcular_frete(5, 100, 'padrao')

tests\test_frete.py:9: AssertionError
test_calcular_frete_expressa

def test_calcular_frete_expressa():
> assert calcular_frete(5, 100, "expressa") == 37.50
E AssertionError: assert None == 37.5
E + where None = calcular_frete(5, 100, 'expressa')

tests\test_frete.py:13: AssertionError

===== short test summary info =====
FAILED tests\test_frete.py::test_calcular_frete_padrao - AssertionError: assert None == 30.0
FAILED tests\test_frete.py::test_calcular_frete_expressa - AssertionError: assert None == 37.5
===== 2 failed, 1 passed in 0.13s =====
PS C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software>
```

Figura 4 – Ciclo 2, etapa Red: os novos testes das modalidades padrão e expressa falham.

```
PS C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software> python -m pytest
===== test session starts =====
platform win32 -- Python 3.12.0, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software
plugins: cov-7.1.0
collected 3 items

tests\test_frete.py ... [100%]

===== 3 passed in 0.03s =====
PS C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software>
```

Figura 5 – Ciclo 2, etapa Green: as três modalidades passam após a implementação dos multiplicadores.

```
Problems Output Debug Console Terminal Ports
PS C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software> python -m pytest
===== test session starts =====
platform win32 -- Python 3.12.0, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software
plugins: cov-7.1.0
collected 8 items

tests\test_frete.py ...FFFFF [100%]

===== FAILURES =====
_____ test_calcular_frete_peso_abaixo_minimo _____

    def test_calcular_frete_peso_abaixo_minimo():
>     with pytest.raises(ValueError):
E     Failed: DID NOT RAISE ValueError

tests\test_frete.py:19: Failed
_____ test_calcular_frete_peso_acima_maximo _____

    def test_calcular_frete_peso_acima_maximo():
>     with pytest.raises(ValueError):
E     Failed: DID NOT RAISE ValueError

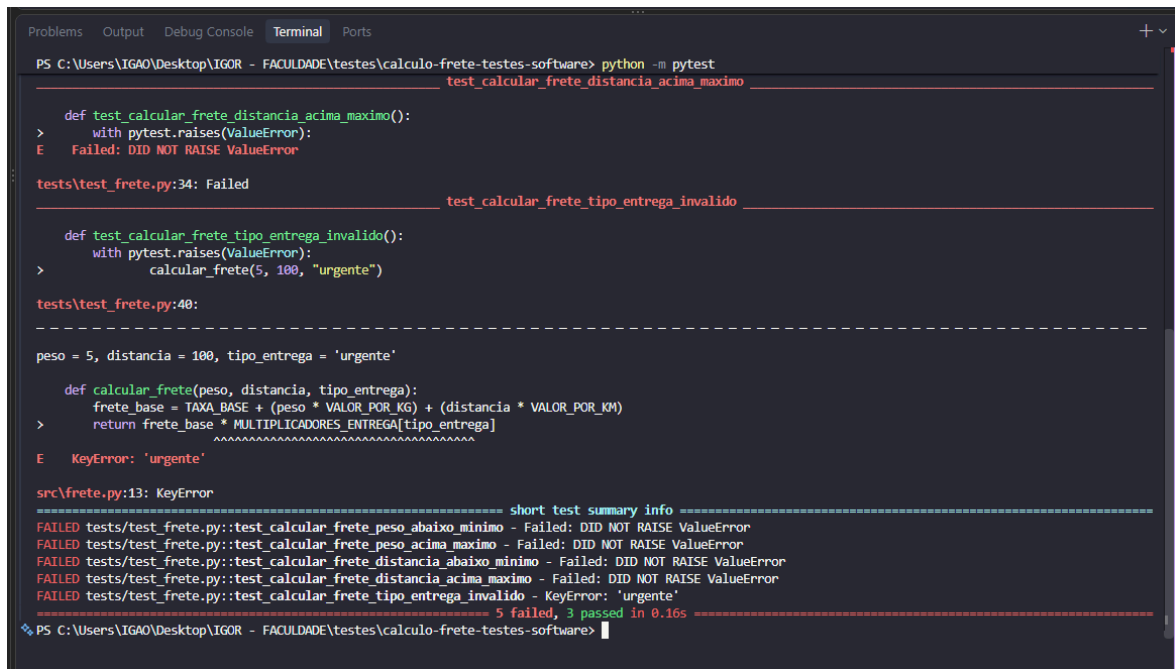
tests\test_frete.py:24: Failed
_____ test_calcular_frete_distancia_abaixo_minimo _____

    def test_calcular_frete_distancia_abaixo_minimo():
>     with pytest.raises(ValueError):
E     Failed: DID NOT RAISE ValueError

tests\test_frete.py:29: Failed
_____ test_calcular_frete_distancia_acima_maximo _____

    def test_calcular_frete_distancia_acima_maximo():
```

Figura 6 – Ciclo 3, etapa Red (parte 1): falhas nas validações de peso e distância.



```
PS C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software> python -m pytest
test_calcular_frete_distancia_acima_maximo

def test_calcular_frete_distancia_acima_maximo():
> with pytest.raises(ValueError):
E Failed: DID NOT RAISE ValueError

tests\test_frete.py:34: Failed
test_calcular_frete_tipo_entrega_invalido

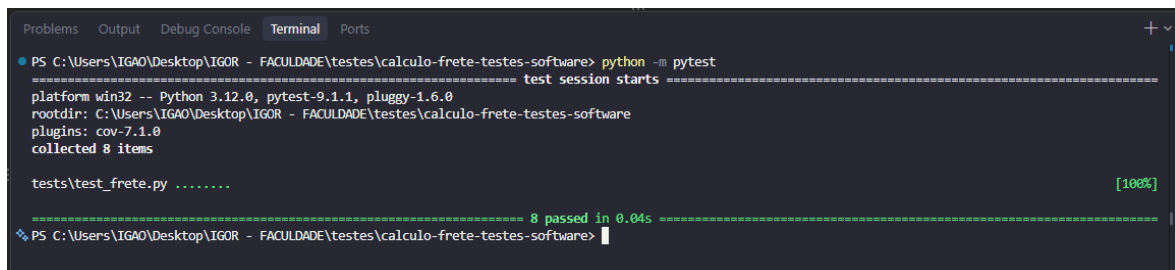
def test_calcular_frete_tipo_entrega_invalido():
with pytest.raises(ValueError):
> calcular_frete(5, 100, "urgente")

tests\test_frete.py:40:
-----
peso = 5, distancia = 100, tipo_entrega = 'urgente'

def calcular_frete(peso, distancia, tipo_entrega):
frete_base = TAXA_BASE + (peso * VALOR_POR_KG) + (distancia * VALOR_POR_KM)
> return frete_base * MULTIPLICADORES_ENTREGA[tipo_entrega]
E KeyError: 'urgente'

src\frete.py:13: KeyError
===== short test summary info =====
FAILED tests/test_frete.py::test_calcular_frete_peso_abaixo_minimo - Failed: DID NOT RAISE ValueError
FAILED tests/test_frete.py::test_calcular_frete_peso_acima_maximo - Failed: DID NOT RAISE ValueError
FAILED tests/test_frete.py::test_calcular_frete_distancia_abaixo_minimo - Failed: DID NOT RAISE ValueError
FAILED tests/test_frete.py::test_calcular_frete_distancia_acima_maximo - Failed: DID NOT RAISE ValueError
FAILED tests/test_frete.py::test_calcular_frete_tipo_entrega_invalido - KeyError: 'urgente'
===== 5 failed, 3 passed in 0.16s =====
❖ PS C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software>
```

Figura 7 – Ciclo 3, etapa Red (parte 2): resumo com cinco falhas, incluindo o tipo de entrega inválido.



```
PS C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software> python -m pytest
===== test session starts =====
platform win32 -- Python 3.12.0, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software
plugins: cov-7.1.0
collected 8 items

tests\test_frete.py ..... [100%]

===== 8 passed in 0.04s =====
❖ PS C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calculo-frete-testes-software>
```

Figura 8 – Ciclo 3, etapa Green: as oito verificações passam após a implementação das validações.

4.3 Automação dos testes unitários

Os testes unitários foram automatizados com o framework pytest. Após a conclusão dos ciclos TDD, a suíte foi reorganizada com o recurso `pytest.mark.parametrize`, permitindo executar o mesmo comportamento com diferentes conjuntos de dados de entrada. Essa abordagem reduz duplicação de código e facilita a aplicação de técnicas como particionamento de equivalência e análise de valor limite.

A parametrização foi utilizada em diferentes grupos de testes, conforme apresentado na tabela a seguir.

Grupo parametrizado	Dados executados	Objetivo
Tipos de entrega	economica, padrao, expressa	Verificar os multiplicadores 1,00; 1,20; 1,50.
Peso - limites válidos	0,1 e 30	Confirmar aceitação dos extremos permitidos.
Peso - valores inválidos	0; 0,09; 30,01; 31	Confirmar rejeição abaixo e acima dos limites.
Distância - limites válidos	1 e 1000	Confirmar aceitação dos extremos permitidos.
Distância - valores inválidos	0 e 1001	Confirmar rejeição fora do intervalo.
Tipo inválido	urgente	Confirmar geração de ValueError.

Como exemplo, o teste das modalidades de entrega executa a mesma função para três conjuntos de dados, alterando apenas o tipo de entrega e o resultado esperado.

```
@pytest.mark.parametrize(
    "tipo_entrega, resultado_esperado",
    [
        ("economica", 25.00),
        ("padrao", 30.00),
        ("expressa", 37.50),
    ],
)
def test_calcular_frete_por_tipo_entrega(
    tipo_entrega,
    resultado_esperado,
):
    assert calcular_frete(5, 100, tipo_entrega) == resultado_esperado
```

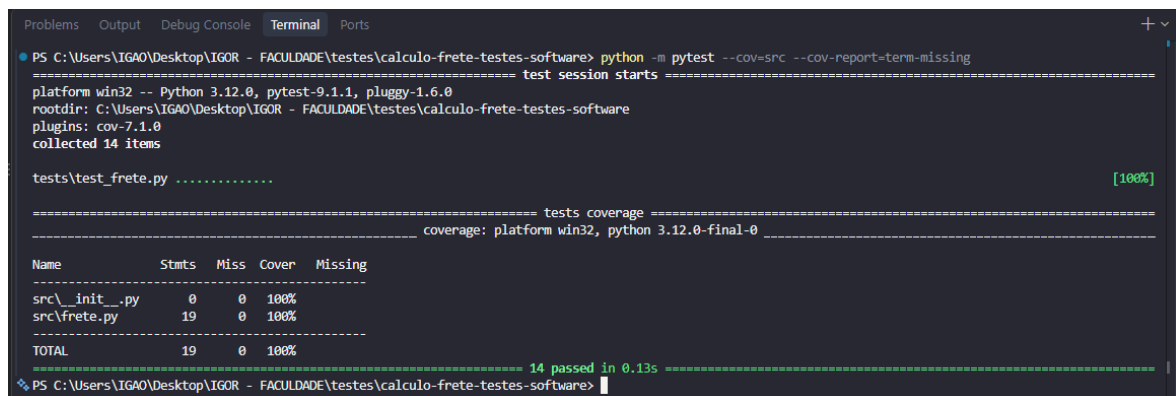
Código 1 — Teste parametrizado para os tipos de entrega.

Nesse teste, o pytest executa a mesma função três vezes. Para peso igual a 5 kg e distância igual a 100 km, o valor-base do frete é de R\$ 25,00. A parametrização permite verificar que a

modalidade econômica retorna R\$ 25,00, a padrão R\$ 30,00 e a expressa R\$ 37,50. A mesma estratégia foi utilizada nos testes dos limites de peso e distância, permitindo verificar as fronteiras definidas nas regras de negócio sem duplicar funções de teste. Ao final, a suíte passou a executar 14 casos automatizados, todos aprovados no estado final do código.

4.4 Cobertura de código

A cobertura foi medida com o plugin pytest-cov por meio do comando "python -m pytest --cov=src --cov-report=term-missing". A execução apresentou 14 testes aprovados e cobertura de 100% no módulo src/frete.py: 19 instruções analisadas, nenhuma instrução não executada e cobertura total de 100%. A Figura 9 registra a execução final utilizada na medição.



```

PS C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calcula-frete-testes-software> python -m pytest --cov=src --cov-report=term-missing
===== test session starts =====
platform win32 -- Python 3.12.0, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calcula-frete-testes-software
plugins: cov-7.1.0
collected 14 items

tests\test_frete.py ..... [100%]

===== tests coverage =====
coverage: platform win32, python 3.12.0-final-0

Name                Stmts  Miss  Cover   Missing
-----
src\__init__.py         0      0   100%
src\frete.py          19      0   100%
TOTAL                  19      0   100%

===== 14 passed in 0.13s =====
PS C:\Users\IGAO\Desktop\IGOR - FACULDADE\testes\calcula-frete-testes-software>

```

Figura 9 – Execução final da suíte: 14 testes aprovados e 100% de cobertura do módulo.

Arquivo	Instruções	Não executadas	Cobertura
src/ __init__.py	0	0	100%
src/frete.py	19	0	100%
TOTAL	19	0	100%

A cobertura de 100% indica que todas as instruções medidas do módulo foram executadas por pelo menos um teste. Entretanto, esse resultado não garante ausência de defeitos nem comprova que todas as combinações possíveis de entrada foram avaliadas. Por esse motivo, a métrica foi analisada em conjunto com particionamento de equivalência, análise de valor limite e testes parametrizados.

5 EXECUÇÃO, DEFEITOS E FERRAMENTAS

5.1 Execução dos testes

A suíte automatizada foi executada inicialmente com a implementação correta, resultando em 14 casos de teste aprovados. Em seguida, foram introduzidos propositalmente dois defeitos independentes no código para avaliar a capacidade dos testes de detectar regressões. Após cada experimento, o código foi restaurado e a suíte voltou a apresentar aprovação total.

Execução	Alteração realizada	Resultado	Status geral
Execução base	Código correto	14 aprovados, 0 reprovados, 0 bloqueados	Aprovado
Defeito 1	Multiplicador da entrega expressa alterado de 1,50 para 1,20	13 aprovados, 1 reprovado, 0 bloqueados	Reprovado
Após correção do defeito 1	Multiplicador restaurado para 1,50	14 aprovados, 0 reprovados, 0 bloqueados	Aprovado
Defeito 2	PESO_MAXIMO alterado de 30 para 31 kg	12 aprovados, 2 reprovados, 0 bloqueados	Reprovado
Após correção do defeito 2	PESO_MAXIMO restaurado para 30 kg	14 aprovados, 0 reprovados, 0 bloqueados	Aprovado

No primeiro defeito, o teste parametrizado da modalidade expressa identificou que `calcular_frete(5, 100, "expressa")` retornava R\$ 30,00, enquanto o resultado esperado era R\$ 37,50. No segundo defeito, os casos parametrizados com pesos 30,01 kg e 31 kg deixaram de gerar `ValueError`, fazendo dois testes falharem. Os resultados demonstram que a suíte consegue detectar alterações incorretas tanto na regra de cálculo quanto nos limites de validação.

5.2 Relatórios de defeito

5.2.1 Defeito 1 - Acréscimo incorreto na entrega expressa

Título	Entrega expressa utiliza multiplicador incorreto e retorna valor menor que o esperado.
---------------	---

Pré-condição	Módulo disponível com o multiplicador da modalidade expressa alterado de 1,50 para 1,20.
Passos para reproduzir	1. Executar <code>calcular_frete(5, 100, "expressa")</code> . 2. Comparar o resultado obtido com o valor esperado segundo as regras de negócio. 3. Executar a suíte com <code>python -m pytest</code> .
Resultado esperado	O frete-base de R\$ 25,00 deve receber acréscimo de 50%, resultando em R\$ 37,50.
Resultado obtido	O sistema retornou R\$ 30,00. A execução apresentou 1 teste reprovado e 13 aprovados.
Severidade	Alta, pois o defeito altera diretamente o valor monetário cobrado do cliente.
Prioridade	Alta, pois a regra central de cálculo da modalidade expressa está incorreta.

5.2.2 Defeito 2 - Limite máximo de peso incorreto

Título	Sistema aceita encomendas acima do peso máximo permitido.
Pré-condição	Constante <code>PESO_MAXIMO</code> alterada propositalmente de 30 para 31 kg.
Passos para reproduzir	1. Executar <code>calcular_frete</code> com peso 30,01 kg, distância 100 km e entrega econômica. 2. Repetir com peso 31 kg. 3. Executar a suíte com <code>python -m pytest</code> .
Resultado esperado	Pesos superiores a 30 kg devem ser rejeitados por meio de <code>ValueError</code> .
Resultado obtido	Os valores 30,01 kg e 31 kg foram aceitos e não lançaram <code>ValueError</code> . A execução apresentou 2 testes reprovados e 12 aprovados.
Severidade	Alta, pois permite processar uma entrada que viola diretamente uma regra de negócio.
Prioridade	Alta, pois a validação de peso define a faixa de operação permitida pelo módulo.

5.3 Comparação de ferramentas de teste

Embora a implementação desenvolvida neste trabalho seja um módulo isolado testado com `pytest`, em uma aplicação completa seriam necessárias ferramentas diferentes para validar interface, API e desempenho. Foram consideradas Cypress, Postman e Apache JMeter.

Ferramenta	Principal finalidade	Aplicação no módulo	Uso escolhido
Cypress	Automação de testes de aplicações web pelo navegador.	Validaria uma eventual interface de cálculo de frete, preenchendo peso, distância e tipo de entrega e verificando o valor exibido.	Testes de interface
Postman	Criação e execução de requisições HTTP para validação de APIs.	Seria utilizado caso calcular_frete fosse exposto por um endpoint, verificando respostas para entradas válidas e inválidas.	Testes de API
Apache JMeter	Testes de carga e desempenho sobre serviços e protocolos.	Permitiria simular grande volume de requisições ao serviço de cálculo de frete para observar tempo de resposta e estabilidade.	Testes de desempenho

Dessa forma, Cypress seria a escolha para testes de interface, Postman para testes de API e JMeter para testes de desempenho. No escopo implementado neste trabalho, pytest permanece como a ferramenta adequada para a automação dos testes unitários do módulo.

6 CONCLUSÃO

O desenvolvimento do módulo de cálculo de frete permitiu aplicar de forma integrada técnicas de planejamento, projeto e automação de testes de software. A definição das regras de negócio orientou a elaboração dos casos de teste e o desenvolvimento do código, incluindo cálculo do frete, modalidades de entrega e validação dos limites de peso e distância.

A aplicação de TDD em três ciclos Red-Green-Refactor evidenciou a evolução gradual do módulo a partir dos testes. Ao final, a suíte automatizada executou 14 casos com pytest, incluindo testes parametrizados voltados a classes de equivalência e valores-limite, alcançando 100% de cobertura de código com pytest-cov.

Apesar disso, 100% de cobertura não garante ausência de defeitos, pois indica apenas que as instruções do código foram executadas. A introdução proposital de dois defeitos demonstrou a importância de combinar cobertura com bons casos de teste: alterações no multiplicador da entrega expressa e no limite máximo de peso foram detectadas porque existiam verificações específicas para resultados esperados e fronteiras das regras de negócio.

Como limitações, o módulo representa um cenário acadêmico simplificado, sem considerar CEP, peso cúbico, transportadoras reais, pedágios, seguro ou integração com serviços externos. Além disso, os testes concentram-se principalmente no nível unitário. Como melhorias futuras, o módulo poderia ser integrado a uma API ou interface, ampliar as regras de precificação e incluir testes de integração, sistema e desempenho.

Dessa forma, o trabalho demonstra que TDD, testes parametrizados, particionamento de equivalência, análise de valor limite, tabela de decisão e cobertura de código se complementam na construção de software mais confiável e com comportamento verificável.